

BDA Course 2026

Lecture 6

BDA Chapter 12

- 12.1 Efficient Gibbs samplers (not part of the course)
- 12.2 Efficient Metropolis jump rules (not part of the course)
- 12.3 Further extensions to Gibbs and Metropolis (not part of the course)
- 12.4 Hamiltonian Monte Carlo (important)
- 12.5 Hamiltonian dynamics for a simple hierarchical model (useful example)
- 12.6 Stan: developing a computing environment (useful intro)

Objectives of this Lecture

- Introduce more advanced and modern MCMC methods
 - Hamiltonian Monte Carlo (HMC) and the No-U-Turn Sampler (NUTS)
- Introduce probabilistic programming languages and in particular Stan

Elevator Pitch

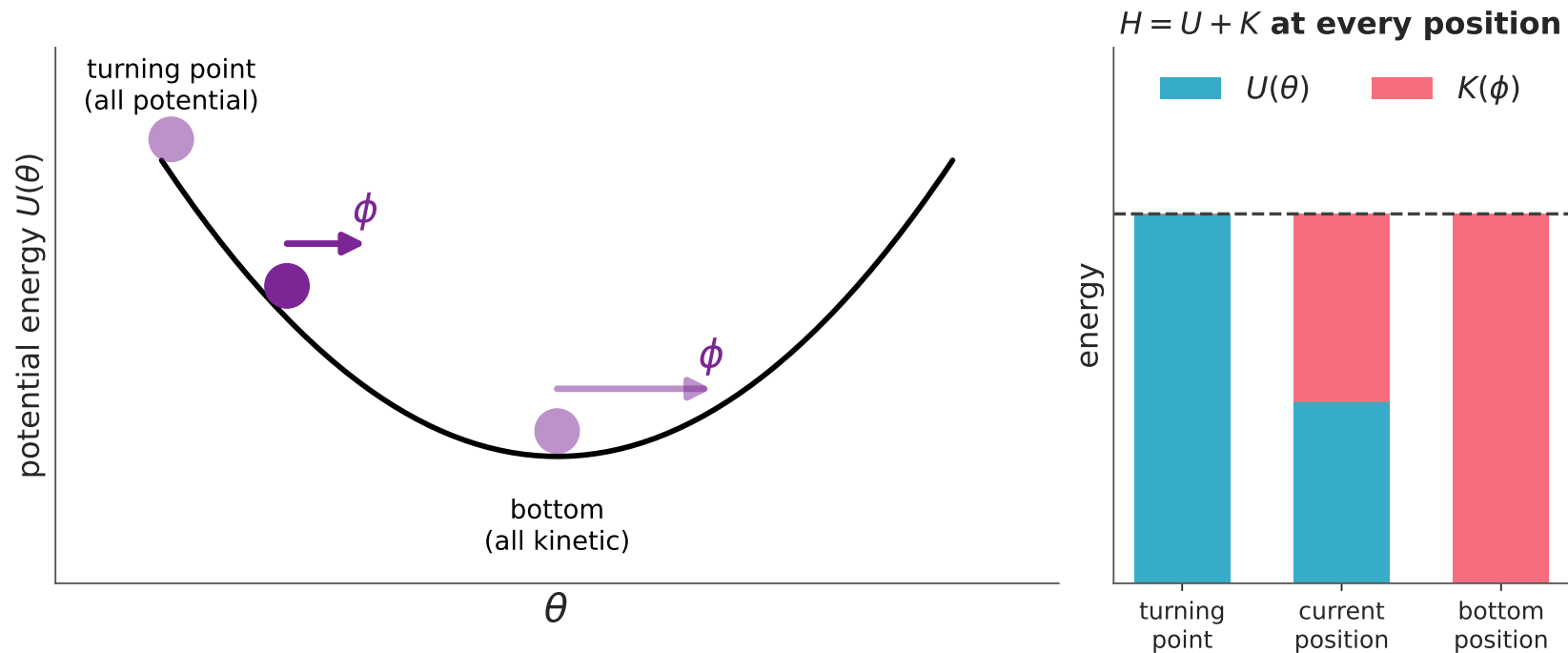
- Hamiltonian Monte Carlo (HMC) is a subclass of MCMC methods that uses gradient information to generate proposals
- In practice this allows HMC to (potentially) explore high-dimensional posterior distributions in an efficient manner
- Why it works: by using gradient information, HMC can propose distant states with high acceptance probability, reducing random-walk behavior and improving sampling efficiency

A little bit of history

- Originally for quantum-chromo-dynamic simulation (Duane et al., 1987)
- Radford Neal started using for Bayesian neural networks in 1990's
- Hoffman and Gelman's (2014) presented the first version of NUTS, making HMC more robust and easier to use in practice
- Nowadays most popular probabilistic programming languages/frameworks use it: Stan, PyMC, NumPyro, Turing.jl, etc
- It is often used as the gold standard for evaluating other inference methods
- What today people informally call "NUTS" has many improvements over the original 2014 version by Hoffman and Gelman.

Hamiltonian dynamics: a physical picture

- Imagine a frictionless particle moving in a 1D “bowl”
 - Position θ : where the particle is
 - Momentum φ : how fast and in which direction it is moving
- This mechanical system can be fully described in terms of:
 - Potential energy $U(\theta)$: depends on the position θ
 - Kinetic energy $K(\varphi)$: depends on how fast the particle moves
- Total energy $H(\theta, \varphi) = U(\theta) + K(\varphi)$ stays constant along the motion



Hamiltonian mechanics

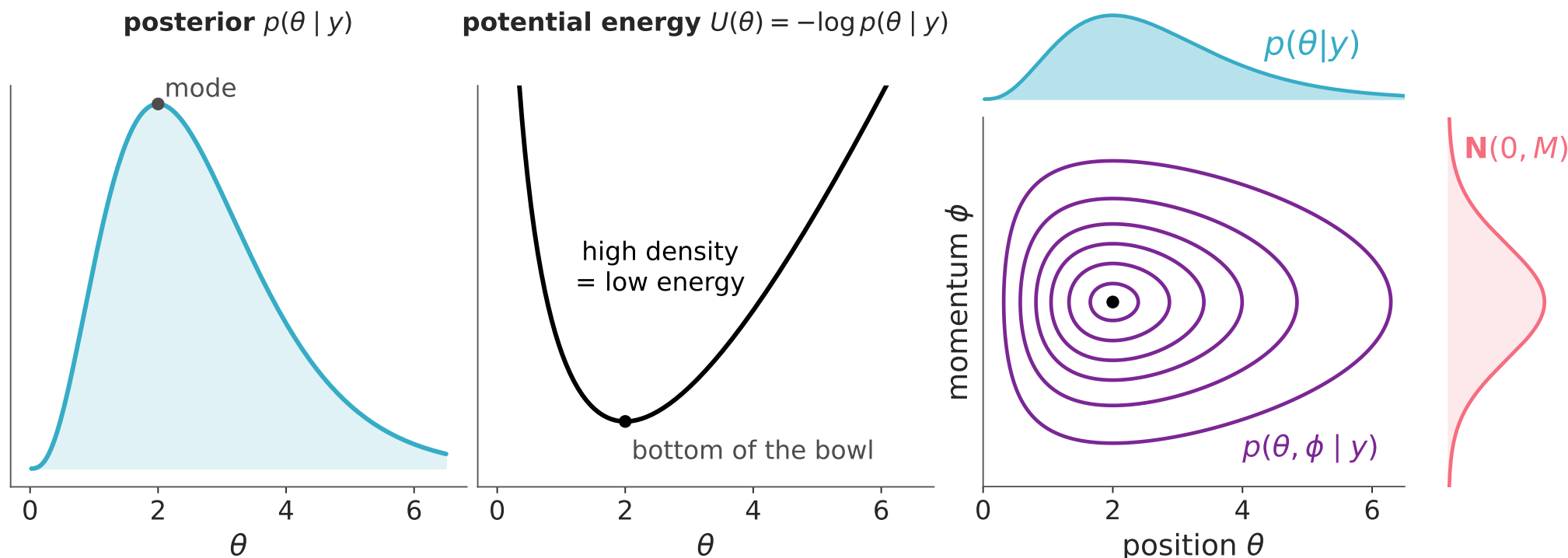
- $H(\theta, \varphi) = U(\theta) + K(\varphi)$ is called the Hamiltonian of the system
- Hamilton's equations describe how position and momentum change in time:

$$\frac{d\theta}{dt} = \frac{\partial H}{\partial \varphi} = M^{-1}\varphi, \quad \frac{d\varphi}{dt} = -\frac{\partial H}{\partial \theta} = -\nabla U(\theta)$$

- Interpretation:
 - Position, θ , changes in proportion to the momentum, with velocity $M^{-1}\varphi$ (M : mass matrix)
 - Momentum, φ , changes in the direction of steepest descent of U , i.e. the force is $-\nabla U(\theta)$
- Key properties, also important for MCMC:
 - Energy, $H(\theta, \varphi)$, is conserved
 - The trajectories are reversible
 - Volume in (θ, φ) space is preserved

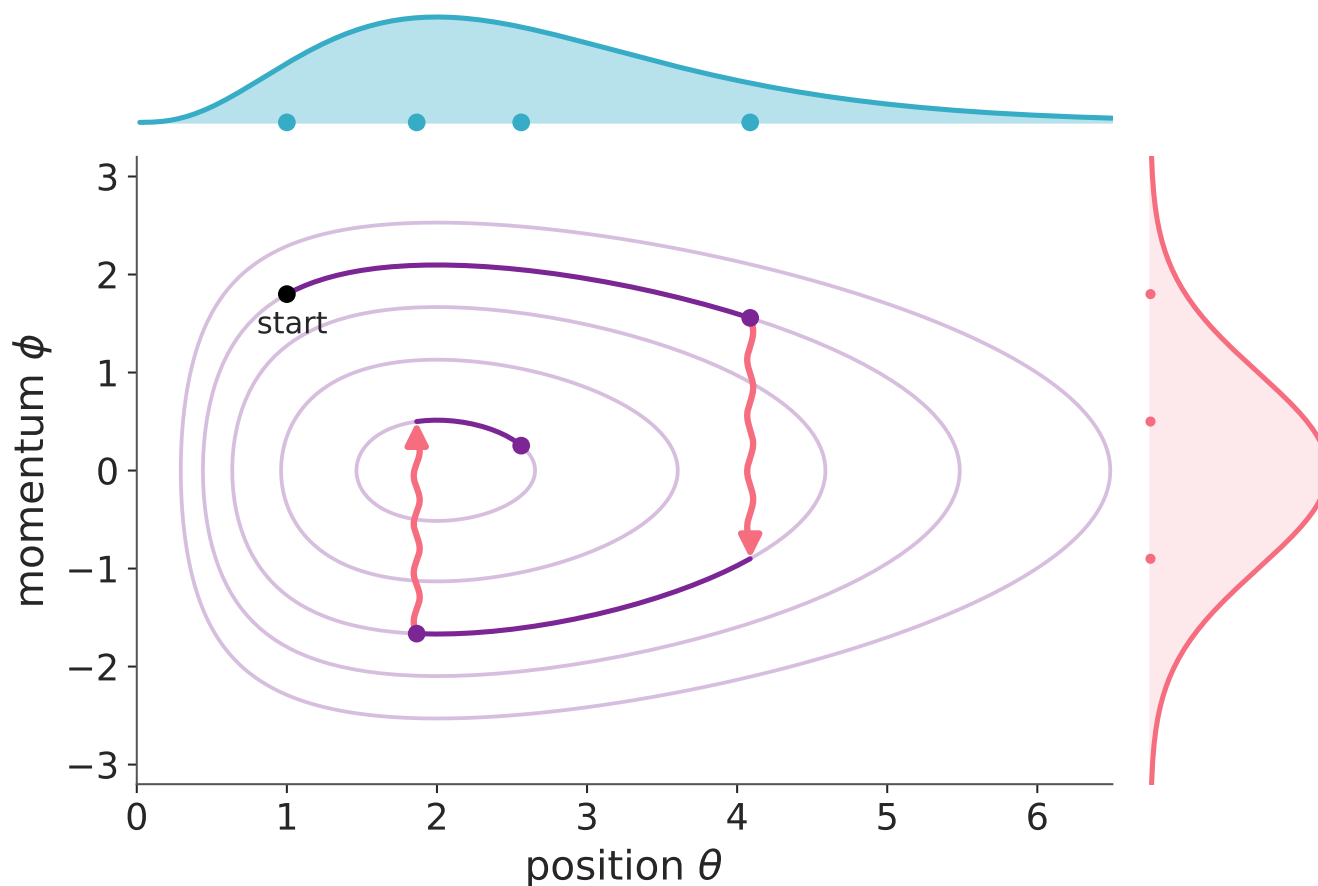
From physics to posterior sampling

- Potential energy: $U(\theta) = -\log p(\theta \mid y) + c$
- Kinetic energy: $K(\varphi) = -\log \mathcal{N}(\varphi \mid 0, M) + c$
- Joint distribution: $p(\theta, \varphi \mid y) \propto \exp(-H(\theta, \varphi))$, with $H = U + K$
- θ and φ are independent, and the marginal of θ is exactly the posterior
- To get samples from $p(\theta \mid y)$, we sample $p(\theta, \varphi \mid y)$ and discard φ



The basic HMC algorithm

1. **Refresh momentum:** draw $\varphi \sim N(0, M)$ (a random “push” in a random direction)
2. **Simulate the dynamics:** follow Hamilton’s equations
3. **Accept or reject** the proposal with a Metropolis step
4. **Repeat** the steps for the desired number of draws



Why this works?

- For an exact simulation H is conserved and every proposal would be accepted
- The proposed value θ^* can be far from θ^{t-1} but still has high acceptance
- In practice the simulation is not exact but we can use Metropolis acceptance to account for errors

Simulating Hamilton's equations

- Except for toy cases, we cannot solve the equations analytically, so we approximate the trajectory with discrete steps of size ε
- The most commonly used method in HMC is the **leapfrog method**. One step:

$$\varphi \leftarrow \varphi - \frac{\varepsilon}{2} \nabla U(\theta)$$

$$\theta \leftarrow \theta + \varepsilon M^{-1} \varphi$$

$$\varphi \leftarrow \varphi - \frac{\varepsilon}{2} \nabla U(\theta)$$

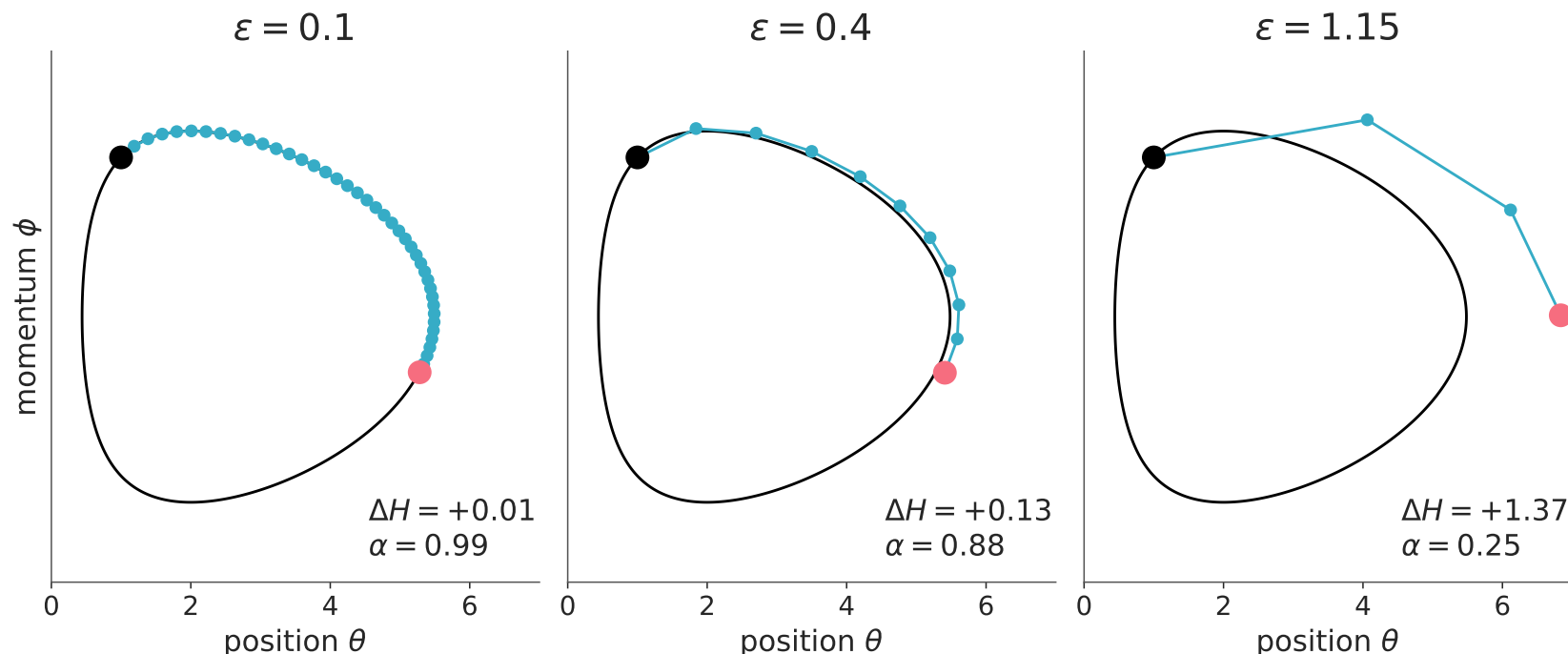
- Repeat for L steps to get the proposal (θ^*, φ^*) .
- It is reversible and volume preserving, and the energy error does not usually grow in time

Leapfrog: step size ε

- The discretization is not exact, so after L steps $H(\theta^*, \varphi^*) \neq H(\theta^{t-1}, \varphi^{t-1})$
- Metropolis step: accept the proposal with probability

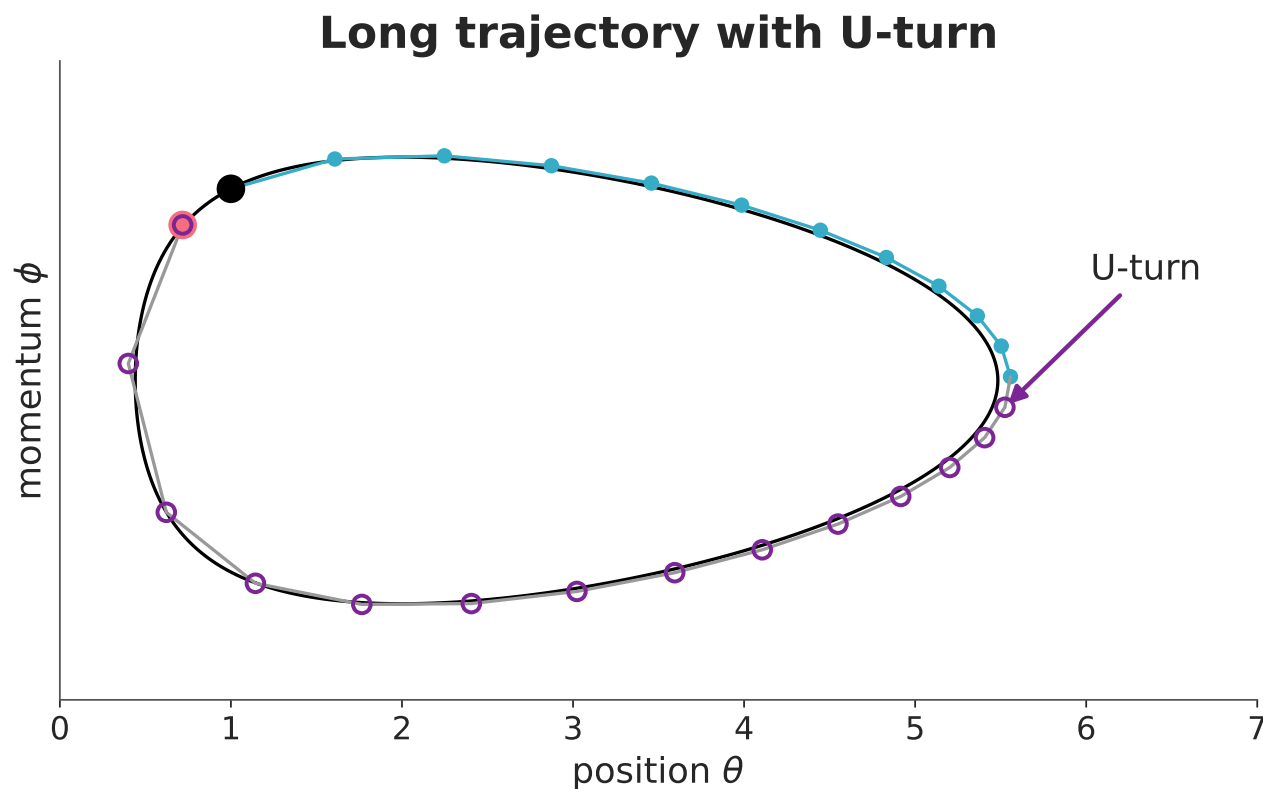
$$\alpha = \min(1, \exp(H(\theta^{t-1}, \varphi^{t-1}) - H(\theta^*, \varphi^*)))$$

- Small ε : small error, so $\alpha \approx 1$, but many steps are needed to move far
- Large ε : fewer steps, larger error, lower α . The simulation may diverge



Leapfrog: number of steps L

- For a fixed ε , the trajectory length is εL
- Too few steps: short moves, which behave like a random walk
- Too many steps: and the trajectory may make a U-turn, wasting computation
- The best ε and L depend on the posterior geometry, so they change from problem to problem



No-U-Turn Sampler (NUTS)

- The NUTS algorithm automatically determines when to stop the trajectory to avoid U-turns
- It makes HMC very easy to use in practice
- It is the de-facto standard for Bayesian inference



NUTS: the U-turn detection

- We need to be careful in how we detect the U-turn to ensure the correctness of the sampler
 1. Start from the current state
 2. Choose at random whether to extend the trajectory forward or backward in time
 3. Run 2^j new leapfrog steps in that direction ($j = 0, 1, 2, \dots$)
 - Internally, a binary tree of states is used to efficiently check for U-turns and preserve reversibility
 4. Check for a U-turn. If found, stop. Otherwise, go to step 2.
 5. Use one of the visited states as the proposed state

NUTS: selecting the proposal

- Original NUTS
 - choose a point along the simulation path and they accept/reject following a Metropolis criterion (details in the original NUTS paper)
- NUTS with multinomial sampling
 - compute the acceptance probability for all points along the simulation path
 - select the point with multinomial sampling
 - more likely to accept a point that is not the previous one
 - No need for a separate Metropolis acceptance step

NUTS: Other details

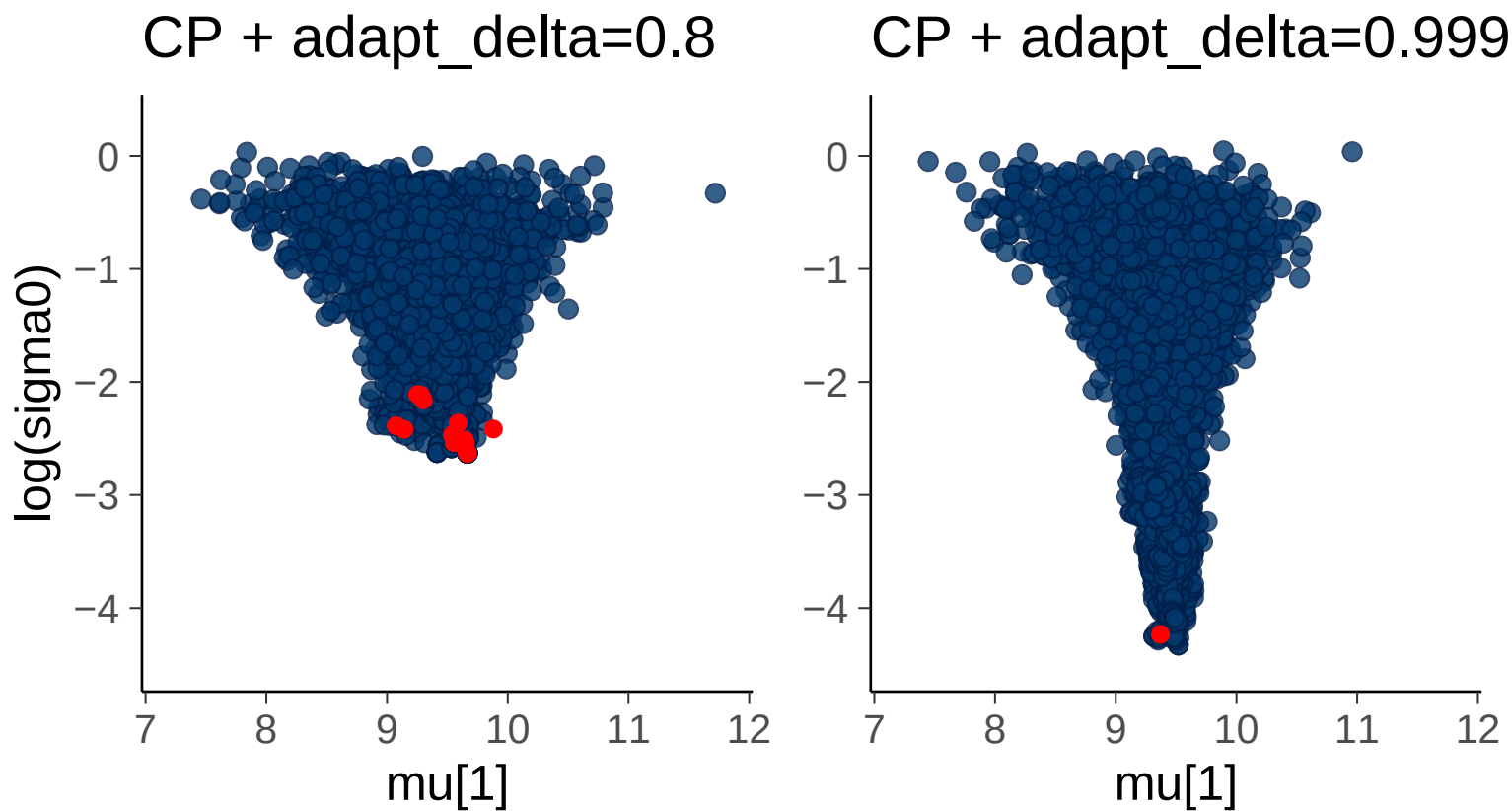
- Besides the U-turn detection we also stop if
 - the trajectory becomes too long (`max_treedepth`)
 - the energy error becomes too large (divergence)
- We also need to choose the step size ε and the mass matrix M
 - We get them adaptively during the warmup phase
 - Mass matrix adapted using an estimate of the posterior covariance
 - Step size adjusted to be as big as possible while keeping discretization error in control (`adapt_delta`)
- After adaptation the algorithm parameters are fixed and some more iterations run to finish the warmup

Divergences

- A divergence happens when the leapfrog simulation becomes unstable and the energy error becomes very large
 - The step size ε is too large for the local geometry of the posterior
- It is an HMC-specific diagnostic: it points to the regions where the sampler has trouble

Divergences

- Divergences may indicate:
 - regions where the log density changes too fast for the step size
 - hence the estimates may be biased



Problematic distributions

- Funnels (next lecture)
 - optimal step size depends on location
- Non-linear dependencies
 - simple mass matrix scaling doesn't help
- Multimodal
 - difficult to move from one mode to another
- Long-tailed with non-finite variance and mean
 - efficiency of exploration is reduced

Recent gradient based samplers

- GPU-friendly alternatives:
 - ChEES-HMC (Hoffman et al., 2021), MEADS (Hoffman & Sountsov, 2022), MALT (Riou-Durand and Vogrinc, 2022; Riou-Durand et al., 2022)
 - The problem with NUTS is that we do not know how large the tree will grow, so each chain needs a different number of leapfrog steps and the others wait for the slowest
- Nutpie (Seyboldt et al., 2026)
 - Better mass matrix adaptation
- WALNUTS (Bou-Rabee et al., 2025)
 - adaptive step size within dynamic simulation

Probabilistic programming language

- A probabilistic programming language (PPL) is a programming language **designed** to describe probabilistic models and then perform **automatic inference** in those models
- A useful PPL should have:
 - inference has to be as automatic as possible
 - fast enough (manual work replaced with automation)
 - access to diagnostics for telling if the automatic inference doesn't work
 - easy workflow and integration with other tools
- Many PPL exist: Stan, PyMC, NumPyro, Turing.jl...

Štrumbelj et al. (2023). Past, Present, and Future of Software for Bayesian Inference. *Statistical Science*, 39(1):46-61. <https://doi.org/10.1214/23-STS907>



Stan PPL, ecosystem and community

- Language, inference engine, user interfaces, documentation, case studies, diagnostics, packages, ...
- Named after Stanislaw Ulam
- Widely popular, 200K+ users in social, biological, and physical sciences, medicine, engineering, and business
- Several full time developers, 40+ developers, 100+ contributors
- R, Python, Julia, Scala, Stata, command line interfaces
- 300+ R packages using Stan



mc-stan.org

Stan language

- Strongly typed domain-specific language for constructing models probabilistics models:
- And then interfaces for standard programming languages like R, Python, Julia... So you can use Stan models within your preferred programming environment.
- Stan transpiles the model to C++, so sampling is fast.
- Stan is Turing complete

Binomial model - Stan code

```
data {  
  int<lower=0> N;           // number of experiments  
  int<lower=0, upper=N> y;  // number of successes  
}
```

- Data type and size are declared
- Stan checks that given data matches type and constraints

Binomial model - Stan code

```
parameters {  
  real<lower=0,upper=1> theta; // parameter of the binomial  
}
```

- Only continuous parameters allowed
 - Discrete parameters can often be integrated out in the model block
- Parameters may have constraints
- Sampling is done in unconstrained space
 - Avoids hard boundaries and hence $-\infty$ log densities
 - Transformations are done automatically (including Jacobian adjustments)
 - e.g. log transformation for `<lower=a>`

Binomial model - Stan code

```
model {  
  theta ~ beta(1, 1);      // prior  
  y ~ binomial(N, theta); // observation model  
}
```

- $\sim$ defines a *distribution statement*
- if y are data, and θ is a parameter, then that term defines log likelihood
- for Stan sampler there is no difference between prior and likelihood, all that matters is the final target

Log density increment statements

```
model {  
  target += beta_lpdf(theta | 1, 1);  
  target += binomial_lpmf(y | N, theta);  
}
```

- left side of `|` denotes what is distributed as, e.g., binomial
- `target` is the log posterior density (Lecture 4 discussed `log`)
- `_lpdf` for continuous, `_lpmf` for discrete distributions (left of `|`)
- You can think of `~` as syntactic sugar for `target +=` statements
- Using `target +=` gives more control and can be useful to express certain models.

CmdStanR

CmdStanR is an R interface for Stan

```
# Load CmdStanR
library(cmdstanr)
options(mc.cores = 1)

# Compile Stan model
mod_bin <- cmdstan_model(stan_file = 'binom.stan')

# Sample from the posterior given the model and data
d_bin <- list(N = 10, y = 7)
fit_bin <- mod_bin$sample(data = d_bin)

# Show summary and access draws
fit_bin$summary()
draws <- fit_bin$draws(format = "df")
```

Difference between proportions

- An experiment was performed to estimate the effect of beta-blockers on mortality of cardiac patients
- A group of patients were randomly assigned to treatment and control groups:
 - out of 674 patients receiving the control, 39 died
 - out of 680 receiving the treatment, 22 died

The model

- θ_1, θ_2 : probability of death in the control and treatment groups
- Uniform priors: $\text{Beta}(1, 1)$
- Deaths in each group are binomial, given the group's death probability

$$\theta_1 \sim \text{Beta}(1, 1)$$

$$\theta_2 \sim \text{Beta}(1, 1)$$

$$y_1 \mid \theta_1 \sim \text{Binomial}(N_1, \theta_1)$$

$$y_2 \mid \theta_2 \sim \text{Binomial}(N_2, \theta_2)$$

- Quantity of interest, the odds ratio (treatment vs. control):

$$\psi = \frac{\frac{\theta_2}{1-\theta_2}}{\frac{\theta_1}{1-\theta_1}}$$

- Data: $N_1 = 674, y_1 = 39$ (control) and $N_2 = 680, y_2 = 22$ (treatment)

The Stan model

```
data {  
  int<lower=0> N1;  
  int<lower=0> y1;  
  int<lower=0> N2;  
  int<lower=0> y2;  
}  
parameters {  
  real<lower=0,upper=1> theta1;  
  real<lower=0,upper=1> theta2;  
}  
model {  
  theta1 ~ beta(1, 1);  
  theta2 ~ beta(1, 1);  
  y1 ~ binomial(N1, theta1);  
  y2 ~ binomial(N2, theta2);  
}  
# Generated quantities is run after the sampling  
generated quantities {  
  real oddsratio;  
  oddsratio = (theta2/(1-theta2))/(theta1/(1-theta1));  
}
```

Difference between proportions

```
d_bin2 <- list(N1 = 674, y1 = 39, N2 = 680, y2 = 22)
mod_bin2 <- cmdstan_model(stan_file = 'binom2.stan')
fit_bin2 <- mod_bin2$sample(data = d_bin2, refresh=1000)
```

> Running MCMC with 4 parallel chains...

```
Chain 1 Iteration:      1 / 2000 [  0%]   (Warmup)
Chain 1 Iteration: 1000 / 2000 [ 50%]   (Warmup)
Chain 1 Iteration: 1001 / 2000 [ 50%]   (Sampling)
Chain 1 Iteration: 2000 / 2000 [100%]   (Sampling)
```

...

All 4 chains finished successfully.

Mean chain execution time: 0.0 seconds.

Total execution time: 0.2 seconds.

Difference between proportions

```
options(posterior.num_args=list(sigfig=2))  
fit_bin2$summary()
```

	variable	mean	median	sd	mad	q5	q95	rhat	ess_bulk	ess_tail
1	lp__	-2.5e+2	-2.5e+2	1.0	0.74	-2.6e+2	-2.5e+2	1.0	1751.	2231.
2	theta1	5.9e-2	5.9e-2	0.0093	0.0093	4.5e-2	7.5e-2	1.0	3189.	2657.
3	theta2	3.4e-2	3.3e-2	0.0069	0.0067	2.3e-2	4.6e-2	1.0	3229.	2163.
4	oddsratio	5.7e-1	5.5e-1	0.16	0.15	3.5e-1	8.7e-1	1.0	2998.	2685.

- lp__ is the log density, ie, same as target

HMC specific diagnostics

```
fit_bin2$diagnostic_summary(diagnostics = c("divergences",  
  "treedepth"))
```

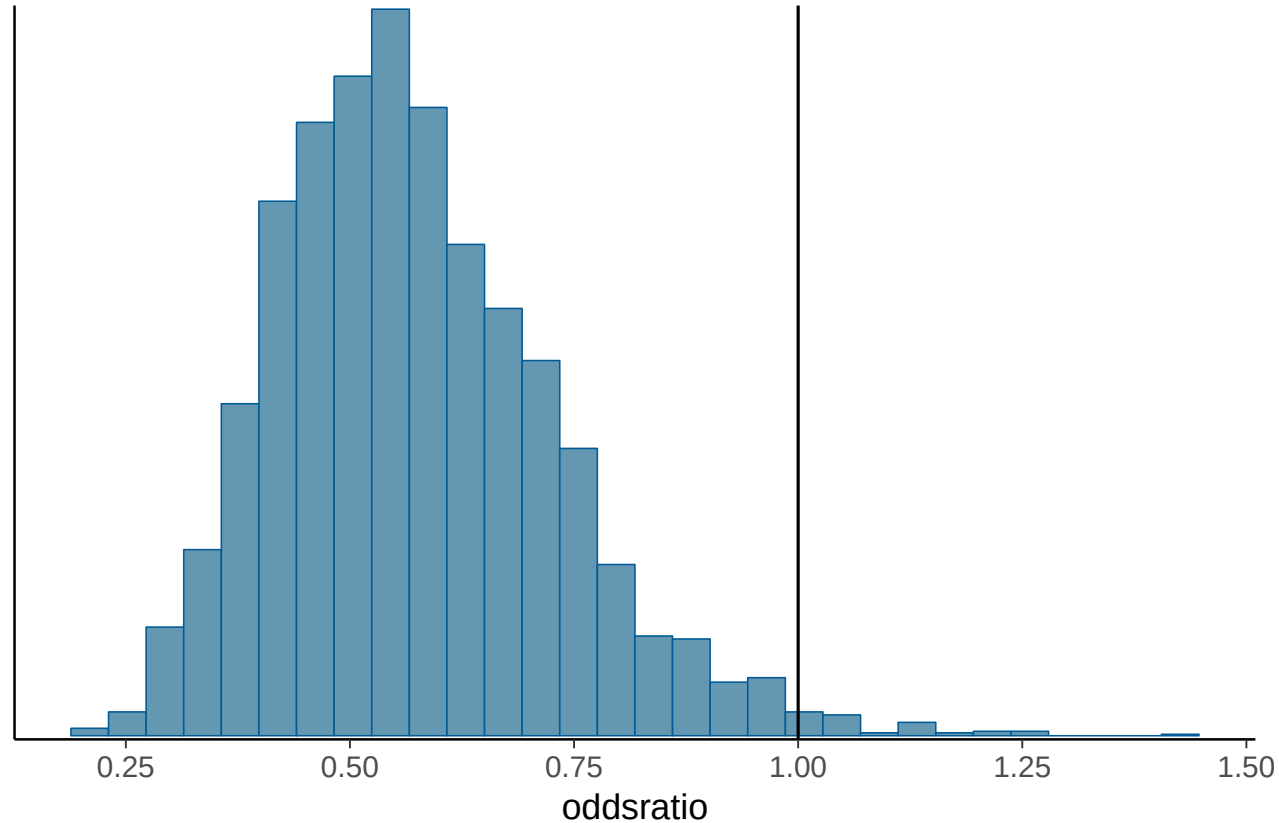
```
$num_divergent  
[1] 0 0 0 0
```

```
$num_max_treedepth  
[1] 0 0 0 0
```

`diagnostic_summary()` includes E-BFMI diagnostic, which I'll skip in this course

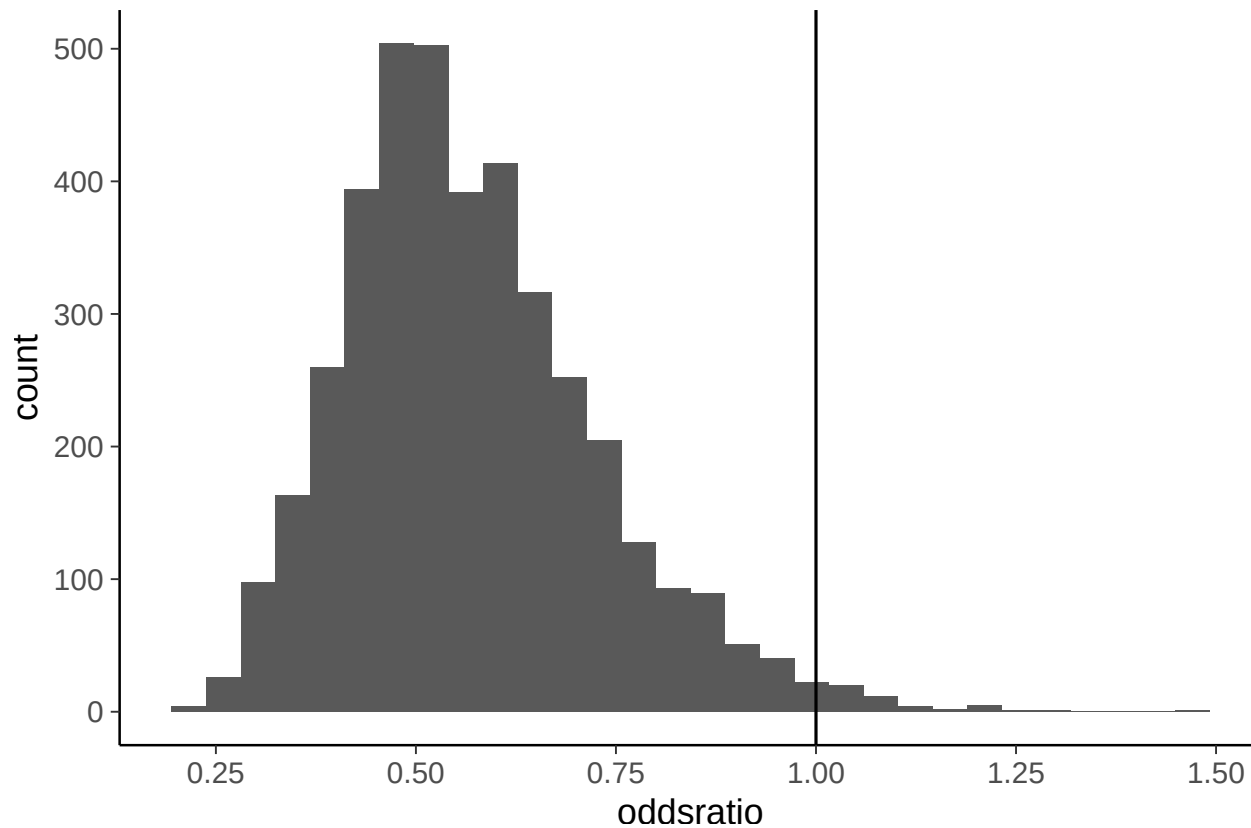
Difference between proportions (bayesplot)

```
draws <- fit_bin2$draws(format = "df")  
mcmc_hist(draws, pars = 'oddsratio') +  
  geom_vline(xintercept = 1) +  
  scale_x_continuous(breaks = c(seq(0.25, 1.5, by=0.25)))
```



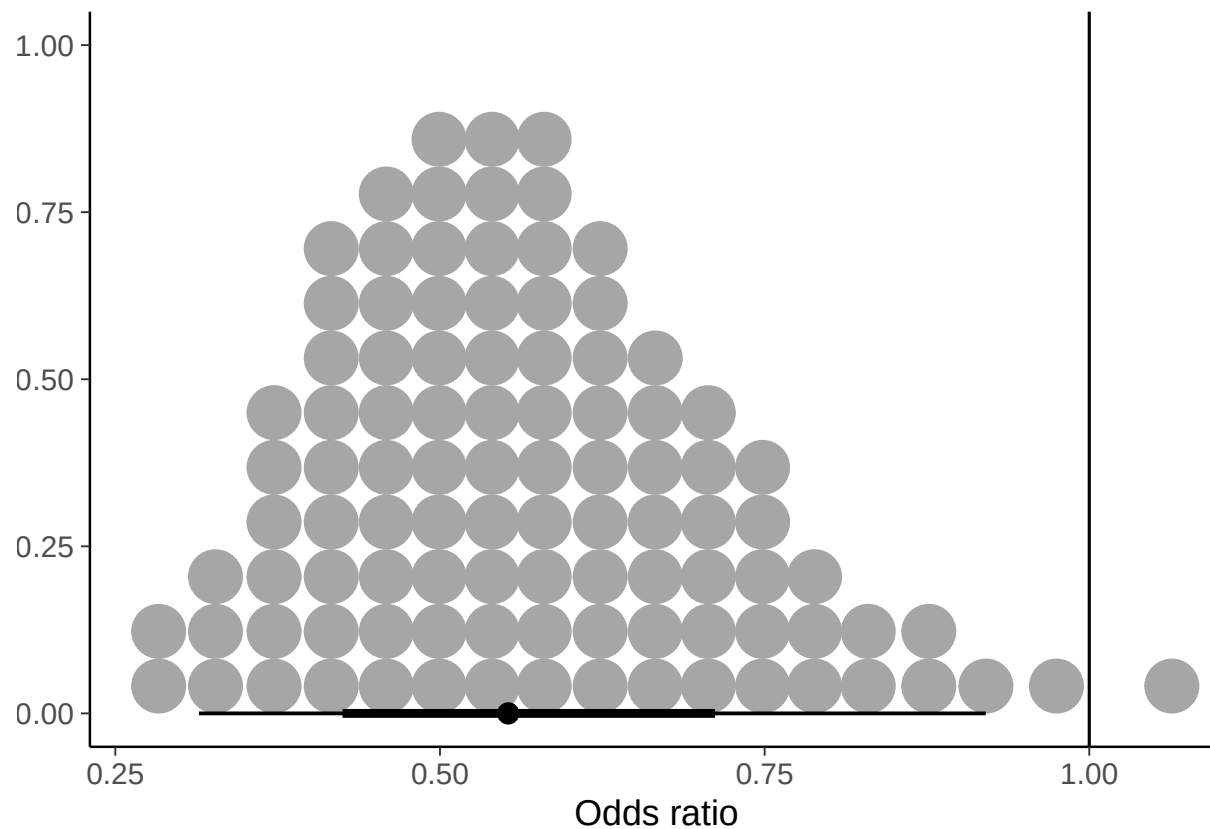
Difference between proportions (ggplot2)

```
draws <- fit_bin2$draws(format = "df")
draws |> ggplot(aes(x=oddsratio)) +
  geom_histogram() +
  geom_vline(xintercept = 1) +
  scale_x_continuous(breaks = c(seq(0.25, 1.5, by=0.25)))
```



Difference between proportions (ggdist dot plot)

```
draws <- fit_bin2$draws(format = "df")
draws |> ggplot(aes(x=oddsratio)) +
  stat_dotsinterval(quantiles = 100) +
  geom_vline(xintercept = 1) +
  scale_x_continuous(breaks = c(seq(0.25, 1.5, by=0.25)))
```



Difference between proportions (probability and MCSE)

Probability (and corresponding MCSE) that $\text{oddsratio} < 1$

```
> draws |>
  mutate_variables(p_oddsratio_lt_1 =
                    as.numeric(oddsratio<1)) |>
  subset_draws("p_oddsratio_lt_1") |>
  summarise_draws(prob=mean, MCSE=mcse_mean)
```

variable	prob	MCSE
p_oddsratio_lt_1	0.99	0.0023

Posterior object formats

Default is `draws_array`

```
> fit_bin2$draws()
```

```
# A draws_array: 1000 iterations, 4 chains, and 4 variables
, , variable = lp__
```

```
      chain
iteration  1    2    3    4
      1 -253 -253 -254 -253
      2 -253 -253 -255 -252
      3 -254 -252 -254 -253
      4 -255 -253 -254 -254
      5 -253 -253 -253 -253
```

```
, , variable = theta1
```

```
      chain
iteration  1    2    3    4
      1 0.054 0.052 0.045 0.049
      2 0.062 0.060 0.070 0.058
```

```
...
```

Posterior object formats

`draws_df` looks prettier and works with `ggplot()`

```
> fit_bin2$draws(format = "df")
```

```
# A draws_df: 1000 iterations, 4 chains, and 4 variables
```

	lp__	theta1	theta2	oddsratio
1	-253	0.054	0.033	0.59
2	-253	0.062	0.035	0.55
3	-254	0.047	0.026	0.54
4	-255	0.049	0.049	0.99
5	-253	0.068	0.035	0.50
6	-253	0.056	0.027	0.47
7	-253	0.071	0.031	0.43
8	-253	0.049	0.036	0.72
9	-253	0.049	0.036	0.72
10	-253	0.063	0.026	0.39

```
# ... with 3990 more draws
```

```
# ... hidden reserved variables {'.chain', '.iteration', '.draw'}
```

posterior object formats

`draws_rvar` makes it easy to compute derived quantities

```
> as_draws_rvars(fit_bin2$draws())
```

```
# A draws_rvars: 1000 iterations, 4 chains, and 4 variables
```

```
$lp__: rvar<1000,4>[1] mean ± sd:
```

```
[1] -253 ± 1
```

```
$theta1: rvar<1000,4>[1] mean ± sd:
```

```
[1] 0.059 ± 0.0093
```

```
$theta2: rvar<1000,4>[1] mean ± sd:
```

```
[1] 0.034 ± 0.0069
```

```
$oddsratio: rvar<1000,4>[1] mean ± sd:
```

```
[1] 0.57 ± 0.16
```

posterior object formats

draws_rvar makes it easy to compute derived quantities

```
> as_draws_rvars(fit_bin2$draws())  
  
# A draws_rvars: 1000 iterations, 4 chains, and 4 variables  
$lp__: rvar<1000,4>[1] mean ± sd:  
[1] -253 ± 1  
  
$theta1: rvar<1000,4>[1] mean ± sd:  
[1] 0.059 ± 0.0093  
  
$theta2: rvar<1000,4>[1] mean ± sd:  
[1] 0.034 ± 0.0069  
  
$oddsratio: rvar<1000,4>[1] mean ± sd:  
[1] 0.57 ± 0.16  
  
> with(draws, (theta2/(1-theta2))/(theta1/(1-theta1)))  
  
rvar<1000,4>[1] mean ± sd:  
[1] 0.5689 ± 0.1577
```

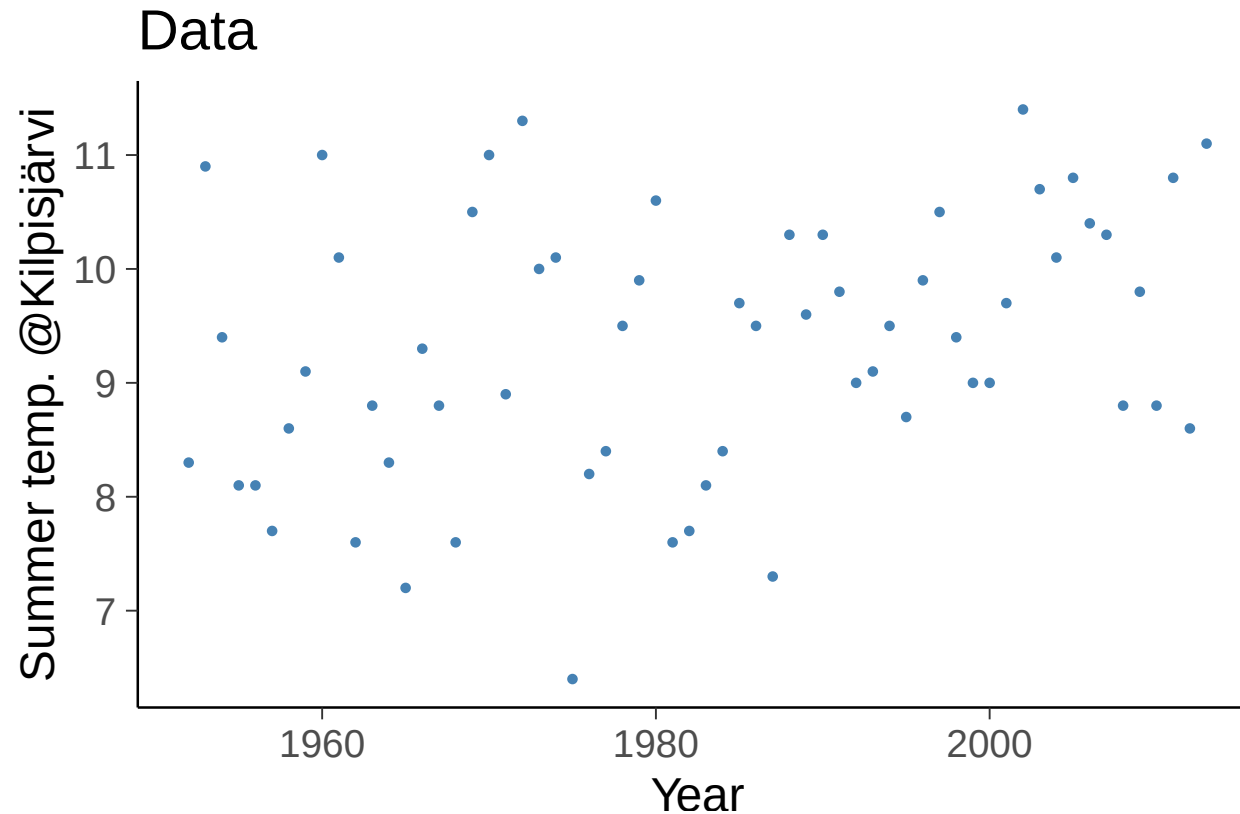
posterior object formats

draws_rvar makes it easy to compute derived quantities

```
> as_draws_rvars(fit_bin2$draws())  
  
# A draws_rvars: 1000 iterations, 4 chains, and 4 variables  
$lp__: rvar<1000,4>[1] mean ± sd:  
[1] -253 ± 1  
  
$theta1: rvar<1000,4>[1] mean ± sd:  
[1] 0.059 ± 0.0093  
  
$theta2: rvar<1000,4>[1] mean ± sd:  
[1] 0.034 ± 0.0069  
  
$oddsratio: rvar<1000,4>[1] mean ± sd:  
[1] 0.57 ± 0.16  
  
> with(draws, (theta2/(1-theta2))/(theta1/(1-theta1)))  
  
rvar<1000,4>[1] mean ± sd:  
[1] 0.5689 ± 0.1577  
  
> draws$oddsratio<1  
  
rvar<1000,4>[1] mean ± sd:  
[1] 0.9865 ± 0.1154
```

Kilpisjärvi summer temperature

- Temperature at Kilpisjärvi in June, July and August from 1952 to 2013
- Is the summer temperature changing?



Normal linear model

```
data {  
  int<lower=0> N;           // number of observations  
  vector[N] x;             // years  
  vector[N] y;             // temperatures  
}  
parameters {  
  real alpha;              // intercept  
  real beta;               // slope  
  real<lower=0> sigma;     // observation model sd  
}  
transformed parameters {  
  vector[N] mu;  
  mu = alpha + beta*x;    // linear model  
}  
model {  
  y ~ normal(mu, sigma);  // observation model  
}
```

Normal linear model

```
data {  
  int<lower=0> N;          // number of observations  
  vector[N] x;  
  vector[N] y;  
}
```

- difference between `vector[N] x` and `array[N] real x`
- only integer arrays: `array[N] int x`

Normal linear model

```
parameters {  
  real alpha;           // intercept  
  real beta;            // slope  
  real<lower=0> sigma;   // observation model sd  
}  
transformed parameters {  
  vector[N] mu;  
  mu = alpha + beta*x;  // linear model  
}
```

- transformed parameters are deterministic transformations of parameters and data

Student- t linear model

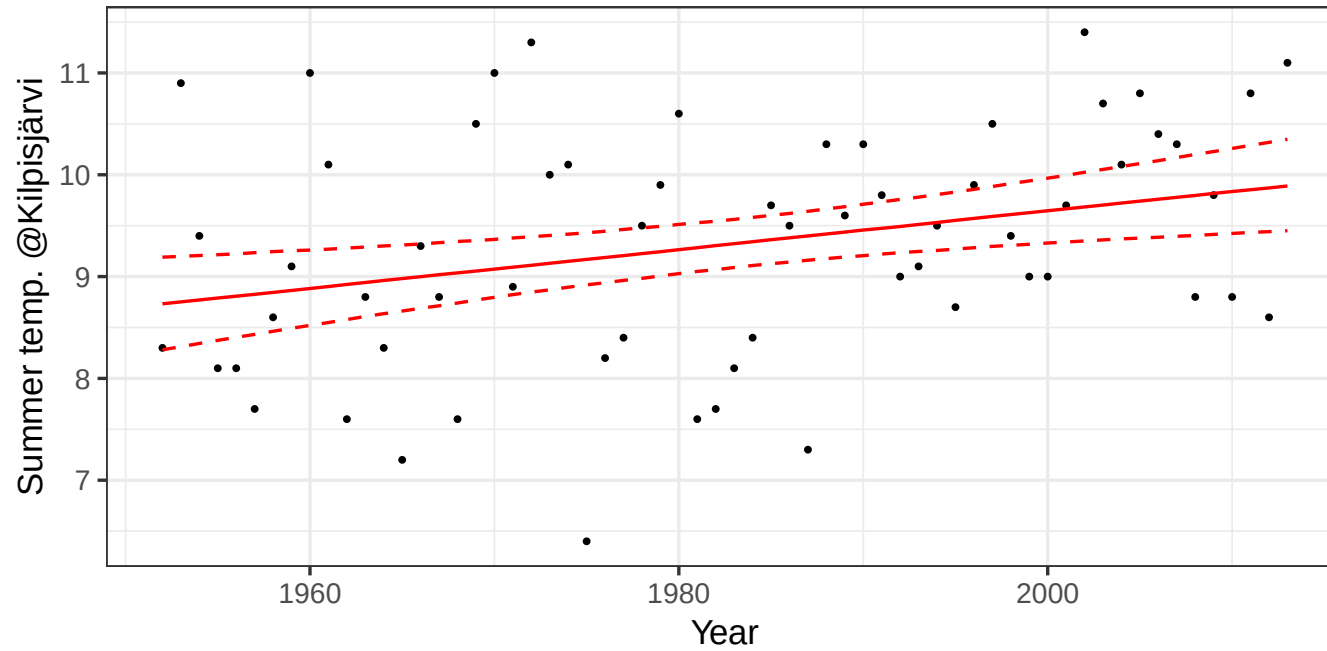
```
...
parameters {
  real alpha;
  real beta;
  real<lower=0> sigma;
  real<lower=1,upper=80> nu;
}
transformed parameters {
  vector[N] mu;
  mu = alpha + beta*x;
}
model {
  nu ~ gamma(2, 0.1);           // prior for nu
  y ~ student_t(nu, mu, sigma); // observation model
}
```

Priors for normal linear model

```
data {  
  int<lower=0> N; // number of observations  
  vector[N] x; //  
  vector[N] y; //  
  real pmualpha; // prior mean for alpha  
  real psalpha;  // prior std for alpha  
  real pmubeta;  // prior mean for beta  
  real psbeta;   // prior std for beta  
}  
...  
transformed parameters {  
  vector[N] mu;  
  mu = alpha + beta*x;  
}  
model {  
  alpha ~ normal(pmualpha, psalpha); // prior for alpha  
  beta ~ normal(pmubeta, psbeta);    // prior for beta  
  y ~ normal(mu, sigma);             // observation model  
}
```

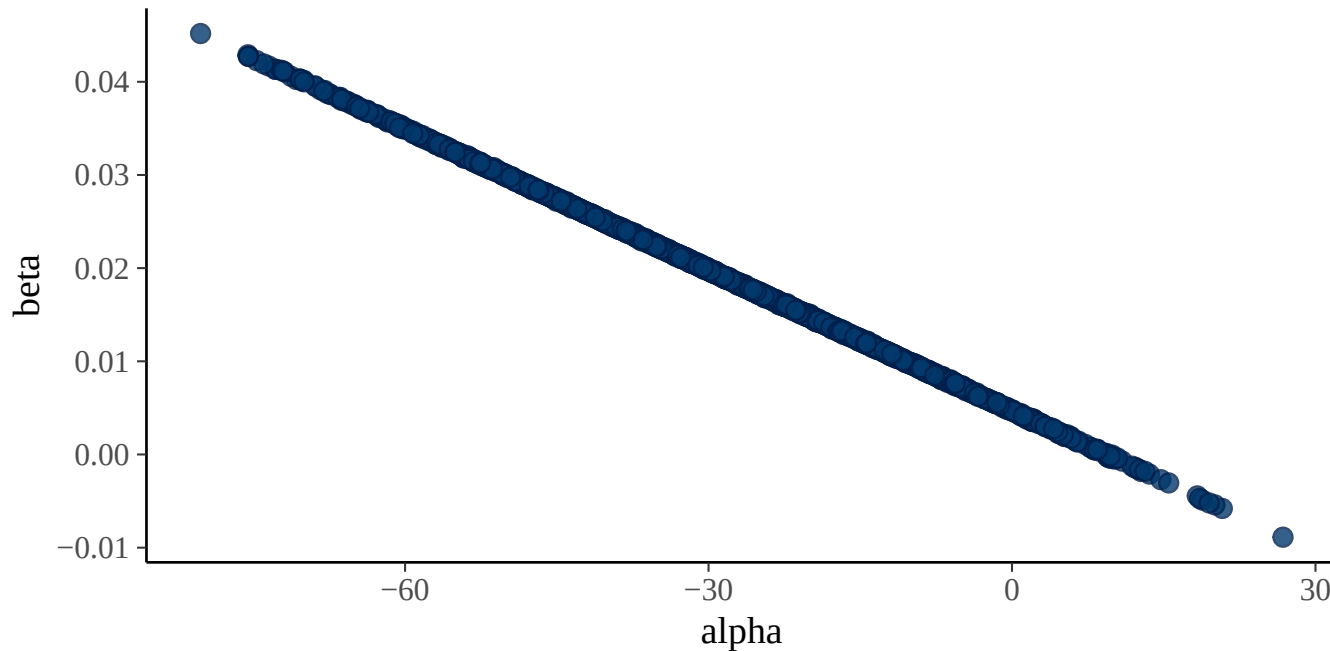
Kilpisjärvi summer temperature

Posterior fit



Kilpisjärvi summer temperature

Posterior draws of alpha and beta

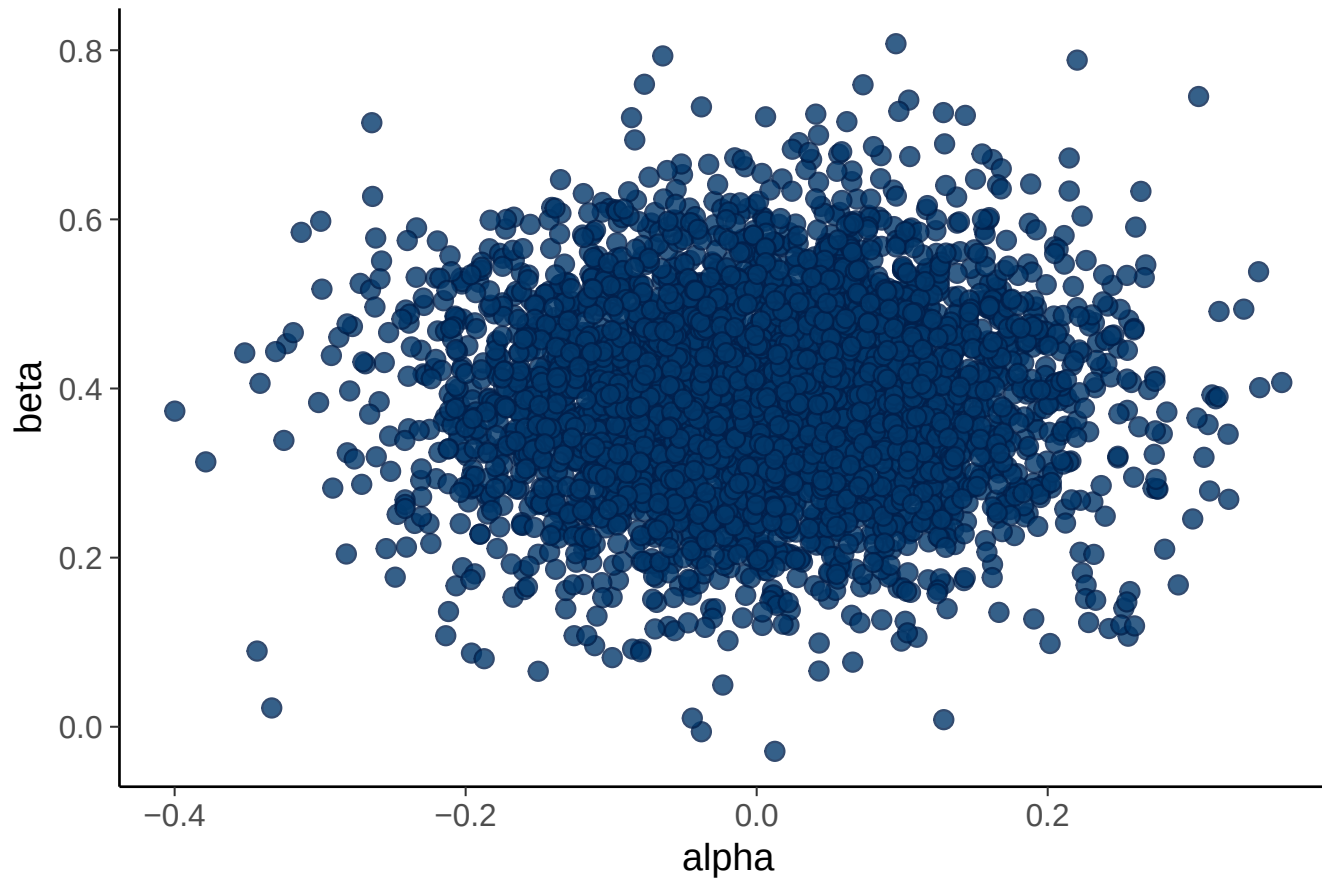


Warning: 1 of 4000 (0.0%) transitions hit the maximum treedepth limit of 10.
See <https://mc-stan.org/misc/warnings> for details.

Hitting maximum treedepth (maximum number of steps) does not invalidate results, but indicates inefficient sampling

Kilpisjärvi summer temperature

Posterior draws of alpha and beta when data is centered



Kilpisjärvi summer temperature

Without centering

```
> fit_lin$summary(variables=c("alpha", "beta"),  
                    default_convergence_measures())
```

variable	rhat	ess_bulk	ess_tail
alpha	1.0	919.	897.
beta	1.0	919.	895.

With centering

```
> fit_lin_std$summary(variables=c("alpha", "beta"),  
                      default_convergence_measures())
```

variable	rhat	ess_bulk	ess_tail
alpha	1.0	3872.	2616.
beta	1.0	3770.	2396.

RStanARM

- RStanARM provides simplified model description with pre-compiled models
 - no need to wait for compilation
 - a restricted set of models

Two group Binomial model:

```
d_bin2 <- data.frame(N = c(674, 680), y = c(39,22), grp2 = c(0,1))
fit_bin2 <- stan_glm(y/N ~ grp2,
                    weights = N,
                    family = binomial(),
                    data = d_bin2)
```

RStanARM

- RStanARM provides simplified model description with pre-compiled models
 - no need to wait for compilation
 - a restricted set of models

Two group Binomial model:

```
d_bin2 <- data.frame(N = c(674, 680), y = c(39, 22), grp2 = c(0, 1))
fit_bin2 <- stan_glm(y/N ~ grp2,
                    weights = N,
                    family = binomial(),
                    data = d_bin2)
```

Normal linear model

```
fit_lin <- stan_glm(temp ~ year,
                   data = d_lin)
```

brms

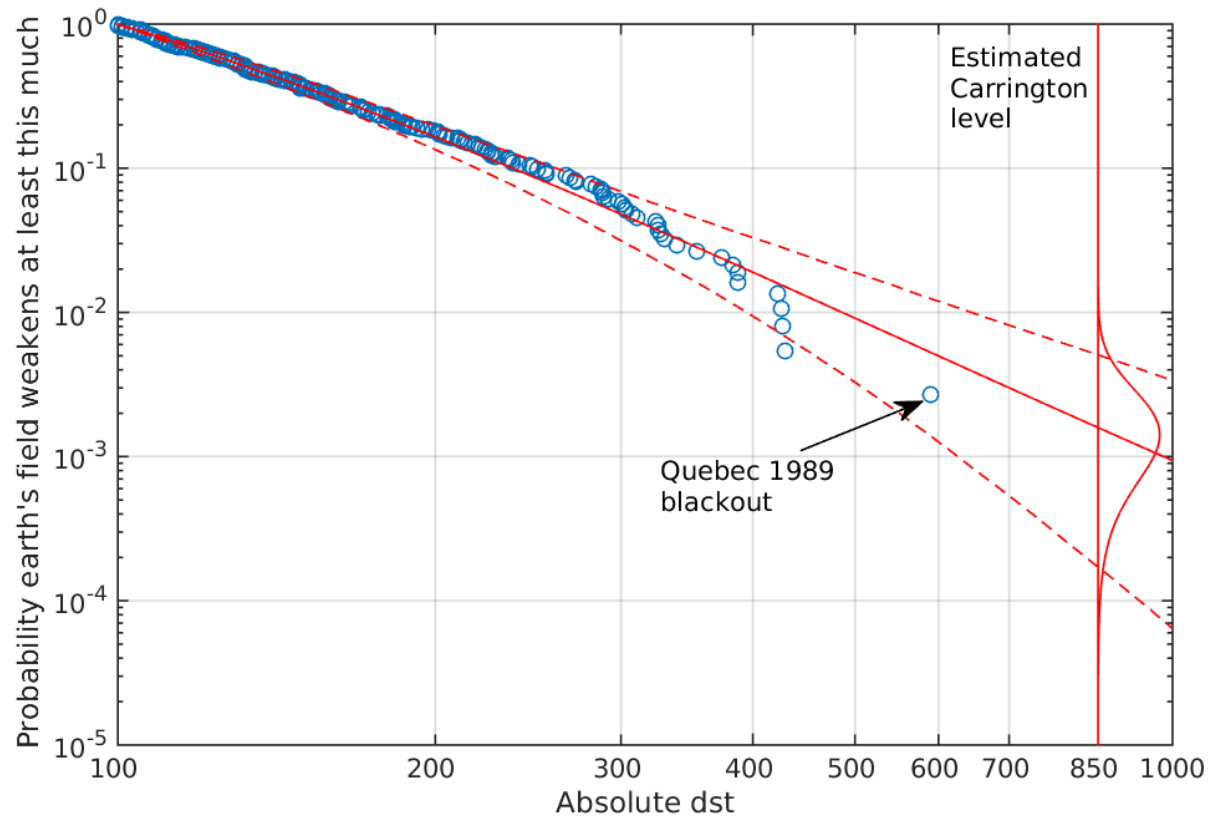
- brms provides simplified model description
 - [+] a larger set of models than RStanARM, but still restricted
 - [-] need to wait for the compilation

```
fit_bin2 <- brm(y | trials(N) ~ grp2,  
               family = binomial(),  
               data = d_bin2)
```

```
fit_lin_t <- brm(temp ~ year,  
                 family = student(),  
                 data = d_lin)
```

Extreme value analysis

- Stan is extensible
 - you can define your own probability distributions and functions
- Geomagnetic storms example



Extreme value analysis

```
data {  
  int<lower=0> N;  
  vector<lower=0>[N] y;  
  int<lower=0> Nt;  
  vector<lower=0>[Nt] yt;  
}  
transformed data {  
  real ymax = max(y);           // pre-compute a useful quantity  
}  
parameters {  
  real<lower=0> sigma;  
  real<lower=-sigma/ymax> k; // constraint can depend on other parameters  
}  
model {  
  y ~ gpareto(k, sigma);        // user defined distribution  
}  
generated quantities {  
  vector[Nt] predccdf = gpareto_ccdf(yt, k, sigma);  
}
```

User defined functions

```
functions {  
  real gpareto_lpdf(vector y, real k, real sigma) {  
    // generalised Pareto log pdf with mu=0  
    // should check and give error if k<0  
    // and max(y)/sigma > -1/k  
    int N;  
    N <- dims(y)[1];  
    if (abs(k) > 1e-15)  
      return -(1+1/k)*sum(log1pv(y*k/sigma)) -N*log(sigma);  
    else  
      return -sum(y/sigma) -N*log(sigma); // limit k->0  
  }  
  vector gpareto_ccdf(vector y, real k, real sigma) {  
    // generalised Pareto log cdf with mu=0  
    // should check and give error if k<0  
    // and max(y)/sigma < -1/k  
    if (abs(k) > 1e-15)  
      return exp((-1/k)*log1pv(y/sigma*k));  
    else  
      return exp(-y/sigma); // limit k->0  
  }  
}
```

Different interfaces

- CmdStanR / CmdStanPy
 - Interface on top of command-line program CmdStan
- RStan / PyStan (deprecated)
 - C++ functions of Stan are called directly from R / Python
 - Higher integration between R/Python and Stan, but maybe more difficult to install due to more requirements of compatible C++ compilers and libraries

Other packages

- R
 - posterior: posterior handling and diagnostics (Lectures 5 and 6)
 - bayesplot and tidybayes: visualization and model checking (Lectures 5, 6, and 8)
 - loo: cross-validation model assessment and comparison (Lecture 9)
 - priorsense: prior and likelihood sensitivity diagnostics (Lecture 12)
 - projpred: projection predictive variable selection (Lecture 12)
 - brms: interface to fit Bayesian generalized (non-)linear multivariate multilevel models using Stan (Lecture 7)
 - marginaleffects: prediction and comparison visualization
- Python
 - ArviZ: visualization, and model checking and assessment
 - Kulprit: projection predictive variable selection
 - Bambi: interface to fit Bayesian generalized (non-)linear multivariate multilevel models using PyMC

Extra material for HMC / NUTS

- An introduction for applied users with good visualizations: Monnahan, Thorson, and Branch (2016) Faster estimation of Bayesian models in ecology using Hamiltonian Monte Carlo. <https://dx.doi.org/10.1111/2041-210X.12681>
- A technical review of why HMC works: Neal (2012). MCMC using Hamiltonian dynamics. <https://arxiv.org/abs/1206.1901>
- The No-U-Turn Sampler: Hoffman and Gelman (2014). The No-U-Turn Sampler: Adaptively Setting Path Lengths in Hamiltonian Monte Carlo. <https://jmlr.csail.mit.edu/papers/v15/hoffman14a.html>
- Multinomial variant of NUTS: Betancourt (2018). A Conceptual Introduction to Hamiltonian Monte Carlo. <https://arxiv.org/abs/1701.02434>
- Seyboldt (2026). Preconditioning Hamiltonian Monte Carlo by minimizing Fisher Divergence. <https://arxiv.org/abs/2603.18845v1>